

```
In [ ]: # Add --break-system-packages at the end of the pip install
# for local installs on OS's like Ubuntu
!git clone https://github.com/ucsd-cse150b-f25/notebooks > /dev/null 2>&1
!mv notebooks/* ./
# After first run, comment the line and rerun
```

```
In [ ]: # Run once and restart your session.
# After restart comment the line and rerun
!pip install -r requirements.txt > /dev/null 2>&1
```

```
In [ ]: # After pip install, run this once, restart your session
# After restart comment the line and rerun
!playwright install chromium > /dev/null 2>&1
```

KNOWLEDGE

The knowledge module covers **Chapter 19: Knowledge in Learning** from Stuart Russel's and Peter Norvig's book *Artificial Intelligence: A Modern Approach*.

Execute the cell below to get started.

```
In [ ]: from knowledge import *
from notebook import psource, pseudocode
from IPython.display import display, Image
import imgkit
```

CONTENTS

- Overview
- Inductive Logic Programming (FOIL)

OVERVIEW

This chapter focuses on methods for generating a model/hypothesis for a domain; however, unlike the learning chapter, here we use prior knowledge to help us learn from new experiences and to find a proper hypothesis.

First-Order Logic

Usually knowledge in this field is represented as **first-order logic**, a type of logic that uses variables and quantifiers in logical sentences. Hypotheses are represented by logical sentences with variables, while examples are logical sentences with set values instead of variables. The goal is to assign a value to a

special first-order logic predicate, called **goal predicate**, for new examples given a hypothesis. We learn this hypothesis by inferring knowledge from some given examples.

Representation

In this module, we use dictionaries to represent examples, with keys being the attribute names and values being the corresponding example values. Examples also have an extra boolean field, 'GOAL', for the goal predicate. A hypothesis is represented as a list of dictionaries. Each dictionary in that list represents a disjunction. Inside these dictionaries/disjunctions we have conjunctions.

For example, say we want to predict if an animal (cat or dog) will take an umbrella given whether or not it rains or the animal wears a coat. The goal value is 'take an umbrella' and is denoted by the key 'GOAL'. An example:

```
{'Species': 'Cat', 'Coat': 'Yes', 'Rain': 'Yes', 'GOAL': True}
```

A hypothesis can be the following:

```
[{'Species': 'Cat'}]
```

which means an animal will take an umbrella if and only if it is a cat.

Consistency

We say that an example `e` is **consistent** with an hypothesis `h` if the assignment from the hypothesis for `e` is the same as `e['GOAL']`. If the above example and hypothesis are `e` and `h` respectively, then `e` is consistent with `h` since `e['Species'] == 'Cat'`. For `e = {'Species': 'Dog', 'Coat': 'Yes', 'Rain': 'Yes', 'GOAL': True}`, the example is no longer consistent with `h`, since the value assigned to `e` is *False* while `e['GOAL']` is *True*.

Inductive Logic Programming (FOIL)

Inductive logic programming (ILP) combines inductive methods with the power of first-order representations, concentrating in particular on the representation of hypotheses as logic programs. The general knowledge-based induction problem is to solve the entailment constraint:

$$Background \wedge Hypothesis \wedge Descriptions \models Classifications$$

for the **unknown** *Hypothesis*, given the *Background* knowledge described by *Descriptions* and *Classifications*.

The first approach to ILP works by starting with a very general rule and gradually specializing it so that it fits the data.

This is essentially what happens in decision-tree learning, where a decision tree is gradually grown until it is consistent with the observations.

To do ILP we use first-order literals instead of attributes, and the *Hypothesis* is a set of clauses (set of first order rules, where each rule is similar to a Horn clause) instead of a decision tree.

The FOIL algorithm learns new rules, one at a time, in order to cover all given positive and negative examples.

More precisely, FOIL contains an inner and an outer while loop.

- **outer loop:** (function **foil()**) add rules until all positive examples are covered.
(each rule is a conjunction of literals, which are chosen inside the inner loop)
- **inner loop:** (function **new_clause()**) add new literals until all negative examples are covered, and some positive examples are covered.
 - In each iteration, we select/add the most promising literal, according to an estimate of its utility. (function **new_literal()**)
 - The evaluation function to estimate utility of adding literal L to a set of rules R is (function **gain()**) :

$$FoilGain(L, R) = t \left(\log_2 \frac{p_1}{p_1 + n_1} - \log_2 \frac{p_0}{p_0 + n_0} \right)$$

where:

p_0 : is the number of positive bindings of rule R

n_0 : is the number of negative bindings of R

p_1 : is the number of positive bindings of rule R'

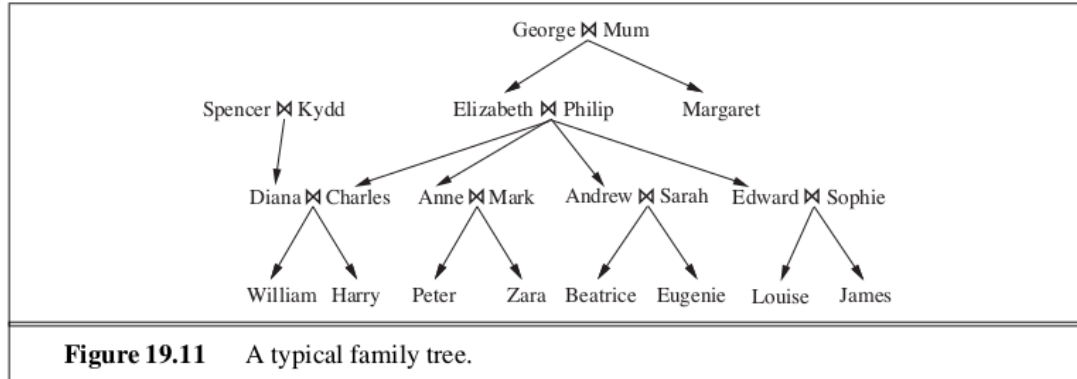
n_0 : is the number of negative bindings of R'

t : is the number of positive bindings of rule R that are still covered after

- Calculate the extended examples for the chosen literal (function **extend_example()**)
(the set of examples created by extending example with each possible constant value for each new variable in literal)
- Finally, the algorithm returns a disjunction of first order rules (= conjunction of literals)

Example Family

Suppose we have the following family relations:



Given some positive and negative examples of the relation 'Parent(x,y)', we want to find a set of rules that satisfies all the examples.

A definition of Parent is $Parent(x,y) \Leftrightarrow Mother(x,y) \vee Father(x,y)$, which is the result that we expect from the algorithm.

```
In [ ]: A, B, C, D, E, F, G, H, I, x, y, z = map(expr, 'ABCDEFGHIXyz')
```

```
In [ ]: small_family = FOILContainer([expr("Mother(Anne, Peter)"),
    expr("Mother(Anne, Zara)"),
    expr("Mother(Sarah, Beatrice)"),
    expr("Mother(Sarah, Eugenie)"),
    expr("Father(Mark, Peter)"),
    expr("Father(Mark, Zara)"),
    expr("Father(Andrew, Beatrice)"),
    expr("Father(Andrew, Eugenie)"),
    expr("Father(Philip, Anne)"),
    expr("Father(Philip, Andrew)"),
    expr("Mother(Elizabeth, Anne)"),
    expr("Mother(Elizabeth, Andrew)"),
    expr("Male(Philip)"),
    expr("Male(Mark)"),
    expr("Male(Andrew)"),
    expr("Male(Peter)"),
    expr("Female(Elizabeth)"),
    expr("Female(Anne)"),
    expr("Female(Sarah)"),
    expr("Female(Zara)"),
    expr("Female(Beatrice)"),
    expr("Female(Eugenie)"),
])
```

```
target = expr('Parent(x, y)')
```

```
examples_pos = [{x: expr('Elizabeth'), y: expr('Anne')},
    {x: expr('Elizabeth'), y: expr('Andrew')},
    {x: expr('Philip'), y: expr('Anne')},
```

```

{x: expr('Philip'), y: expr('Andrew')},
{x: expr('Anne'), y: expr('Peter')},
{x: expr('Anne'), y: expr('Zara')},
{x: expr('Mark'), y: expr('Peter')},
{x: expr('Mark'), y: expr('Zara')},
{x: expr('Andrew'), y: expr('Beatrice')},
{x: expr('Andrew'), y: expr('Eugenie')},
{x: expr('Sarah'), y: expr('Beatrice')},
{x: expr('Sarah'), y: expr('Eugenie')}
examples_neg = [{x: expr('Anne'), y: expr('Eugenie')},
{x: expr('Beatrice'), y: expr('Eugenie')},
{x: expr('Mark'), y: expr('Elizabeth')},
{x: expr('Beatrice'), y: expr('Philip')}]

```

```

In [ ]: # run the FOIL algorithm
clauses = small_family.foil([examples_pos, examples_neg], target)
print (clauses)

```

Indeed the algorithm returned the rule:

$$Parent(x, y) \Leftrightarrow Mother(x, y) \vee Father(x, y)$$

Suppose that we have some positive and negative results for the relation 'GrandParent(x,y)' and we want to find a set of rules that satisfies the examples. One possible set of rules for the relation $Grandparent(x, y)$ could be:

$$\begin{aligned}
 Grandparent(x, y) \Leftrightarrow & [\exists z \text{ } Mother(x, z) \wedge Mother(z, y)] \\
 & \vee [\exists z \text{ } Mother(x, z) \wedge Father(z, y)] \\
 & \vee [\exists z \text{ } Father(x, z) \wedge Mother(z, y)] \\
 & \vee [\exists z \text{ } Father(x, z) \wedge Father(z, y)] .
 \end{aligned}$$

Or, if *Background* included the sentence

$$Parent(x, y) \Leftrightarrow [Mother(x, y) \vee Father(x, y)] \text{ then:}$$

$$Grandparent(x, y) \Leftrightarrow \exists z \text{ } Parent(x, z) \wedge Parent(z, y)$$

```

In [ ]: target = expr('Grandparent(x, y)')

examples_pos = [{x: expr('Elizabeth'), y: expr('Peter')},
{x: expr('Elizabeth'), y: expr('Zara')},
{x: expr('Elizabeth'), y: expr('Beatrice')},
{x: expr('Elizabeth'), y: expr('Eugenie')},
{x: expr('Philip'), y: expr('Peter')},
{x: expr('Philip'), y: expr('Zara')},
{x: expr('Philip'), y: expr('Beatrice')},
{x: expr('Philip'), y: expr('Eugenie')}]
examples_neg = [{x: expr('Anne'), y: expr('Eugenie')},
{x: expr('Beatrice'), y: expr('Eugenie')},
{x: expr('Elizabeth'), y: expr('Andrew')},
{x: expr('Elizabeth'), y: expr('Anne')},
{x: expr('Elizabeth'), y: expr('Mark')},
{x: expr('Elizabeth'), y: expr('Sarah')},
{x: expr('Philip'), y: expr('Anne')},

```

```

{x: expr('Philip'), y: expr('Andrew')},
{x: expr('Anne'), y: expr('Peter')},
{x: expr('Anne'), y: expr('Zara')},
{x: expr('Mark'), y: expr('Peter')},
{x: expr('Mark'), y: expr('Zara')},
{x: expr('Andrew'), y: expr('Beatrice')},
{x: expr('Andrew'), y: expr('Eugenie')},
{x: expr('Sarah'), y: expr('Beatrice')},
{x: expr('Mark'), y: expr('Elizabeth')},
{x: expr('Beatrice'), y: expr('Philip')},
{x: expr('Peter'), y: expr('Andrew')},
{x: expr('Zara'), y: expr('Mark')},
{x: expr('Peter'), y: expr('Anne')},
{x: expr('Zara'), y: expr('Eugenie')},
]

```

```

clauses = small_family.foil([examples_pos, examples_neg], target)

print(clauses)

```

Indeed the algorithm returned the rule:

$Grandparent(x, y) \Leftrightarrow \exists v \text{ Parent}(x, v) \wedge \text{Parent}(v, y)$

Example Network

Suppose that we have the following directed graph and we want to find a rule that describes the reachability between two nodes (Reach(x,y)).

Such a rule could be recursive, since y can be reached from x if and only if there is a sequence of adjacent nodes from x to y:

$$Reach(x, y) \Leftrightarrow \begin{cases} Conn(x, y), & \text{(if there is a directed edge from x to y)} \\ \vee \exists z \text{ } Reach(x, z) \wedge Reach(z, y) \end{cases}$$

In []:

```

"""
A          H
| \       / |
|  \     /  |
v   v   v   v
B  D-->E-->G-->I
|  /      |
| /       |
vv      v
C        F
"""

small_network = FOILContainer([expr("Conn(A, B)"),
                                expr("Conn(A, D)"),
                                expr("Conn(B, C)"),
                                expr("Conn(D, C)"),
                                expr("Conn(D, E)"),
                                expr("Conn(E, F)"),
                                expr("Conn(E, G)"),
                                expr("Conn(G, I)"),

```

```
expr("Conn(H, G)",  
expr("Conn(H, I)"]])
```

```
In [ ]: target = expr('Reach(x, y)')  
examples_pos = [{x: A, y: B},  
                {x: A, y: C},  
                {x: A, y: D},  
                {x: A, y: E},  
                {x: A, y: F},  
                {x: A, y: G},  
                {x: A, y: I},  
                {x: B, y: C},  
                {x: D, y: C},  
                {x: D, y: E},  
                {x: D, y: F},  
                {x: D, y: G},  
                {x: D, y: I},  
                {x: E, y: F},  
                {x: E, y: G},  
                {x: E, y: I},  
                {x: G, y: I},  
                {x: H, y: G},  
                {x: H, y: I}]  
nodes = {A, B, C, D, E, F, G, H, I}  
examples_neg = [example for example in [{x: a, y: b} for a in nodes for b in nodes  
                if example not in examples_pos]  
clauses = small_network.foil([examples_pos, examples_neg], target)  
print(clauses)
```

The algorithm produced something close to the recursive rule:

$$Reach(x, y) \Leftrightarrow [Conn(x, y)] \vee [\exists z \, Reach(x, z) \wedge Reach(z, y)]$$

This happened because the size of the example is small.

```
In [ ]:
```

CURRENT-BEST LEARNING

Overview

In **Current-Best Learning**, we start with a hypothesis and we refine it as we iterate through the examples. For each example, there are three possible outcomes: the example is consistent with the hypothesis, the example is a **false positive** (real value is false but got predicted as true) and the example is a **false negative** (real value is true but got predicted as false). Depending on the outcome we refine the hypothesis accordingly:

- Consistent: We do not change the hypothesis and move on to the next example.

- False Positive: We **specialize** the hypothesis, which means we add a conjunction.
- False Negative: We **generalize** the hypothesis, either by removing a conjunction or a disjunction, or by adding a disjunction.

When specializing or generalizing, we should make sure to not create inconsistencies with previous examples. To avoid that caveat, backtracking is needed. Thankfully, there is not just one specialization or generalization, so we have a lot to choose from. We will go through all the specializations/generalizations and we will refine our hypothesis as the first specialization/generalization consistent with all the examples seen up to that point.

Pseudocode

```
In [ ]: pseudocode('Current-Best-Learning')
```

Implementation

As mentioned earlier, examples are dictionaries (with keys being the attribute names) and hypotheses are lists of dictionaries (each dictionary is a disjunction). Also, in the hypothesis, we denote the *NOT* operation with an exclamation mark (!).

We have functions to calculate the list of all specializations/generalizations, to check if an example is consistent/false positive/false negative with a hypothesis. We also have an auxiliary function to add a disjunction (or operation) to a hypothesis, and two other functions to check consistency of all (or just the negative) examples.

You can read the source by running the cell below:

```
In [ ]: psource(current_best_learning, specializations, generalizations)
```

You can view the auxiliary functions in the [knowledge module](#). A few notes on the functionality of some of the important methods:

- `specializations` : For each disjunction in the hypothesis, it adds a conjunction for values in the examples encountered so far (if the conjunction is consistent with all the examples). It returns a list of hypotheses.
- `generalizations` : It adds to the list of hypotheses in three phases. First it deletes disjunctions, then it deletes conjunctions and finally it adds a

disjunction.

- `add_or` : Used by `generalizations` to add an *or operation* (a disjunction) to the hypothesis. Since the last example is the problematic one which wasn't consistent with the hypothesis, it will model the new disjunction to that example. It creates a disjunction for each combination of attributes in the example and returns the new hypotheses consistent with the negative examples encountered so far. We do not need to check the consistency of positive examples, since they are already consistent with at least one other disjunction in the hypotheses' set, so this new disjunction doesn't affect them. In other words, if the value of a positive example is negative under the disjunction, it doesn't matter since we know there exists a disjunction consistent with the example.

Since the algorithm stops searching the specializations/generalizations after the first consistent hypothesis is found, usually you will get different results each time you run the code.

Examples

We will take a look at two examples. The first is a trivial one, while the second is a bit more complicated (you can also find it in the book).

Earlier, we had the "animals taking umbrellas" example. Now we want to find a hypothesis to predict whether or not an animal will take an umbrella. The attributes are `Species`, `Rain` and `Coat`. The possible values are `[Cat, Dog]`, `[Yes, No]` and `[Yes, No]` respectively. Below we give seven examples (with `GOAL` we denote whether an animal will take an umbrella or not):

```
In [ ]: animals_umbrellas = [
    {'Species': 'Cat', 'Rain': 'Yes', 'Coat': 'No', 'GOAL': True},
    {'Species': 'Cat', 'Rain': 'Yes', 'Coat': 'Yes', 'GOAL': True},
    {'Species': 'Dog', 'Rain': 'Yes', 'Coat': 'Yes', 'GOAL': True},
    {'Species': 'Dog', 'Rain': 'Yes', 'Coat': 'No', 'GOAL': False},
    {'Species': 'Dog', 'Rain': 'No', 'Coat': 'No', 'GOAL': False},
    {'Species': 'Cat', 'Rain': 'No', 'Coat': 'No', 'GOAL': False},
    {'Species': 'Cat', 'Rain': 'No', 'Coat': 'Yes', 'GOAL': True}
]
```

Let our initial hypothesis be `[{'Species': 'Cat'}]`. That means every cat will be taking an umbrella. We can see that this is not true, but it doesn't matter since we will refine the hypothesis using the Current-Best algorithm. First, let's see how that initial hypothesis fares to have a point of reference.

```
In [ ]: initial_h = [{'Species': 'Cat'}]
```

```
for e in animals_umbrellas:  
    print(guess_value(e, initial_h))
```

We got 5/7 correct. Not terribly bad, but we can do better. Lets now run the algorithm and see how that performs in comparison to our current result.

```
In [ ]: h = current_best_learning(animals_umbrellas, initial_h)  
  
for e in animals_umbrellas:  
    print(guess_value(e, h))
```

We got everything right! Let's print our hypothesis:

```
In [ ]: print(h)
```

If an example meets any of the disjunctions in the list, it will be `True`, otherwise it will be `False`.

Let's move on to a bigger example, the "Restaurant" example from the book. The attributes for each example are the following:

- Alternative option (`Alt`)
- Bar to hang out/wait (`Bar`)
- Day is Friday (`Fri`)
- Is hungry (`Hun`)
- How much does it cost (`Price` , takes values in [,\$,\$\$])
- How many patrons are there (`Pat` , takes values in [None, Some, Full])
- Is raining (`Rain`)
- Has made reservation (`Res`)
- Type of restaurant (`Type` , takes values in [French, Thai, Burger, Italian])
- Estimated waiting time (`Est` , takes values in [0-10, 10-30, 30-60, >60])

We want to predict if someone will wait or not (Goal = WillWait). Below we show twelve examples found in the book.

Example	Input Attributes										Goal
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>	<i>WillWait</i>
x₁	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>0–10</i>	<i>y₁ = Yes</i>
x₂	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>30–60</i>	<i>y₂ = No</i>
x₃	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Some</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>0–10</i>	<i>y₃ = Yes</i>
x₄	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Thai</i>	<i>10–30</i>	<i>y₄ = Yes</i>
x₅	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>>60</i>	<i>y₅ = No</i>
x₆	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Italian</i>	<i>0–10</i>	<i>y₆ = Yes</i>
x₇	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>0–10</i>	<i>y₇ = No</i>
x₈	<i>No</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Thai</i>	<i>0–10</i>	<i>y₈ = Yes</i>
x₉	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>>60</i>	<i>y₉ = No</i>
x₁₀	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>Italian</i>	<i>10–30</i>	<i>y₁₀ = No</i>
x₁₁	<i>No</i>	<i>No</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>0–10</i>	<i>y₁₁ = No</i>
x₁₂	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>30–60</i>	<i>y₁₂ = Yes</i>

Figure 18.3 Examples for the restaurant domain.

With the function `r_example` we will build the dictionary examples:

```
In [ ]: def r_example(Alt, Bar, Fri, Hun, Pat, Price, Rain, Res, Type, Est, GOAL):
        return {'Alt': Alt, 'Bar': Bar, 'Fri': Fri, 'Hun': Hun, 'Pat': Pat,
                'Price': Price, 'Rain': Rain, 'Res': Res, 'Type': Type, 'Est': Est,
                'GOAL': GOAL}
```

In code:

```
In [ ]: restaurant = [
    r_example('Yes', 'No', 'No', 'Yes', 'Some', '$$$', 'No', 'Yes', 'French', '0-10', 'Yes'),
    r_example('Yes', 'No', 'No', 'Yes', 'Full', '$', 'No', 'No', 'Thai', '30-60', 'No'),
    r_example('No', 'Yes', 'No', 'No', 'Some', '$', 'No', 'No', 'Burger', '0-10', 'Yes'),
    r_example('Yes', 'No', 'Yes', 'Yes', 'Full', '$', 'Yes', 'No', 'Thai', '10-30', 'Yes'),
    r_example('Yes', 'No', 'Yes', 'No', 'Full', '$$$', 'No', 'Yes', 'French', '>60', 'No'),
    r_example('No', 'Yes', 'No', 'Yes', 'Some', '$$', 'Yes', 'Yes', 'Italian', '0-10', 'Yes'),
    r_example('No', 'Yes', 'No', 'No', 'None', '$', 'Yes', 'No', 'Burger', '0-10', 'No'),
    r_example('No', 'No', 'No', 'Yes', 'Some', '$$', 'Yes', 'Yes', 'Thai', '0-10', 'Yes'),
    r_example('No', 'Yes', 'Yes', 'No', 'Full', '$', 'Yes', 'No', 'Burger', '>60', 'No'),
    r_example('Yes', 'Yes', 'Yes', 'Yes', 'Full', '$$$', 'No', 'Yes', 'Italian', '10-30', 'No'),
    r_example('No', 'No', 'No', 'No', 'None', '$', 'No', 'No', 'Thai', '0-10', 'No'),
    r_example('Yes', 'Yes', 'Yes', 'Yes', 'Full', '$', 'No', 'No', 'Burger', '30-60', 'Yes')
]
```

Say our initial hypothesis is that there should be an alternative option and lets run the algorithm.

```
In [ ]: initial_h = [{'Alt': 'Yes'}]
h = current_best_learning(restaurant, initial_h)
for e in restaurant:
    print(guess_value(e, h))
```

The predictions are correct. Let's see the hypothesis that accomplished that:

```
In [ ]: print(h)
```

It might be quite complicated, with many disjunctions if we are unlucky, but it will always be correct, as long as a correct hypothesis exists.

```
In [ ]:
```

VERSION-SPACE LEARNING

Overview

Version-Space Learning is a general method of learning in logic based domains. We generate the set of all the possible hypotheses in the domain and then we iteratively remove hypotheses inconsistent with the examples. The set of remaining hypotheses is called **version space**. Because hypotheses are being removed until we end up with a set of hypotheses consistent with all the examples, the algorithm is sometimes called **candidate elimination** algorithm.

After we update the set on an example, all the hypotheses in the set are consistent with that example. So, when all the examples have been parsed, all the remaining hypotheses in the set are consistent with all the examples. That means we can pick hypotheses at random and we will always get a valid hypothesis.

Pseudocode

```
In [ ]: pseudocode('Version-Space-Learning')
```

Implementation

The set of hypotheses is represented by a list and each hypothesis is represented by a list of dictionaries, each dictionary a disjunction. For each example in the given examples we update the version space with the function `version_space_update`. In the end, we return the version-space.

Before we can start updating the version space, we need to generate it. We do that with the `all_hypotheses` function, which builds a list of all the possible hypotheses (including hypotheses with disjunctions). The function works like this: first it finds the possible values for each attribute (using `values_table`), then it builds all the attribute combinations (and adds them to the hypotheses set) and finally it builds the combinations of all the disjunctions (which in this case are the hypotheses build by the attribute combinations).

You can read the code for all the functions by running the cells below:

```
In [ ]: psource(version_space_learning, version_space_update)
```

```
In [ ]: psource(all_hypotheses, values_table)
```

```
In [ ]: psource(build_attr_combinations, build_h_combinations)
```

Example

Since the set of all possible hypotheses is enormous and would take a long time to generate, we will come up with another, even smaller domain. We will try and predict whether we will have a party or not given the availability of pizza and soda. Let's do it:

```
In [ ]: party = [  
    {'Pizza': 'Yes', 'Soda': 'No', 'GOAL': True},  
    {'Pizza': 'Yes', 'Soda': 'Yes', 'GOAL': True},  
    {'Pizza': 'No', 'Soda': 'No', 'GOAL': False}  
]
```

Even though it is obvious that no-pizza no-party, we will run the algorithm and see what other hypotheses are valid.

```
In [ ]: V = version_space_learning(party)  
for e in party:  
    guess = False  
    for h in V:  
        if guess_value(e, h):  
            guess = True  
            break  
  
print(guess)
```

The results are correct for the given examples. Let's take a look at the version space:

```
In [ ]: print(len(V))  
  
print(V[5])  
print(V[10])  
  
print(['Pizza': 'Yes'] in V)
```

There are almost 1000 hypotheses in the set. You can see that even with just two attributes the version space is very large.

Our initial prediction is indeed in the set of hypotheses. Also, the two other random hypotheses we got are consistent with the examples (since they both include the "Pizza is available" disjunction).

Minimal Consistent Determination

This algorithm is based on a straightforward attempt to find the simplest determination consistent with the observations. A determination $P \supset Q$ says that if any examples match on P, then they must also match on Q. A determination is therefore consistent with a set of examples if every pair that matches on the predicates on the left-hand side also matches on the goal predicate.

Pseudocode

Lets look at the pseudocode for this algorithm

```
In [ ]: pseudocode('Minimal-Consistent-Det')
```

You can read the code for the above algorithm by running the cells below:

```
In [ ]: psource(minimal_consistent_det)
```

```
In [ ]: psource(consistent_det)
```

Example:

We already know that no-pizza-no-party but we will still check it through the `minimal_consistent_det` algorithm.

```
In [ ]: print(minimal_consistent_det(party, {'Pizza', 'Soda'}))
```

We can also check it on some other example. Let's consider the following example :

```
In [ ]: conductance = [
    {'Sample': 'S1', 'Mass': 12, 'Temp': 26, 'Material': 'Cu', 'Size': 3, 'Color': 'Red'},
    {'Sample': 'S1', 'Mass': 12, 'Temp': 100, 'Material': 'Cu', 'Size': 3, 'Color': 'Blue'},
    {'Sample': 'S2', 'Mass': 24, 'Temp': 26, 'Material': 'Cu', 'Size': 6, 'Color': 'Red'},
    {'Sample': 'S3', 'Mass': 12, 'Temp': 26, 'Material': 'Pb', 'Size': 2, 'Color': 'Red'},
    {'Sample': 'S3', 'Mass': 12, 'Temp': 100, 'Material': 'Pb', 'Size': 2, 'Color': 'Blue'},
    {'Sample': 'S4', 'Mass': 18, 'Temp': 100, 'Material': 'Pb', 'Size': 3, 'Color': 'Red'},
    {'Sample': 'S4', 'Mass': 18, 'Temp': 100, 'Material': 'Pb', 'Size': 3, 'Color': 'Blue'},
    {'Sample': 'S5', 'Mass': 24, 'Temp': 100, 'Material': 'Pb', 'Size': 4, 'Color': 'Red'},
    {'Sample': 'S6', 'Mass': 36, 'Temp': 26, 'Material': 'Pb', 'Size': 6, 'Color': 'Red'}
]
```

Now, we check the `minimal_consistent_det` algorithm on the above example:

```
In [ ]: print(minimal_consistent_det(conductance, {'Mass', 'Temp', 'Material', 'Size'}
```

```
In [ ]: print(minimal_consistent_det(conductance, {'Mass', 'Temp', 'Size'}))
```

```
In [ ]: # Download your notebook, and upload it as mdp.pdf  
!jupyter nbconvert --to webpdf --embed-images knowledge_F0IL.ipynb
```